ROCm-SDXL AI Creation Workbench

Track 1 — Multimodal AI Content Creation Tools

AMD Radeon Pro W7900D | ROCm 7.2.1 | SDXL 1.0 Base

SDXL 1024×1024 | 20 steps | avg 8.02 s | peak 31.21 GB VRAM

Submission includes: source code · README · workflows · error logs · demo outputs

1. Project Background

This project builds an open, reproducible AI image creation workbench on the AMD ROCm ecosystem, centered on SDXL 1.0 Base. It addresses three industry pain points:

- Native AI drawing tools are optimized for NVIDIA CUDA only. AMD ROCm deployments suffer from library-loading conflicts, missing operators, and model-decoding errors, making the barrier to entry extremely high for ordinary developers.
- Container cloud environments lose models and dependencies on restart. There is no standardized persistent deployment workflow for ROCm-based AI services.
- No integrated ROCm tool combines professional node-based debugging (ComfyUI) with rapid visual drawing (A1111 WebUI).

Positioning: an out-of-the-box, deeply AMD-optimized multimodal image generation workbench built on SDXL, bundled with complete deployment scripts and troubleshooting guides.

Exploration: Z-Image-Turbo lightweight generation model was investigated and its workflow was scaffolded. VAE weight mismatch prevents full inference; the integration is reserved for future iterations.

2. Target Users & Application Scenarios

Users:

Target users:

- AI creators and student developers with AMD GPUs or ROCm cloud servers.
- Game / anime studios needing batch illustration and character concept art.
- Algorithm learners who require SDXL workflow debugging and API batch-generation testbeds.

Scenarios:

Application scenarios:

- General high-fidelity realistic image generation (SDXL Base 1.0).
- Anime / illustration batch creation (Animagine XL 4.0).
- Remote cloud demonstration: local browser access via SSH tunnel, no public port exposure.
- Educational reproducibility: one-click scripts for zero-experience ROCm environment setup.

3. System Architecture

Four-layer architecture, fully leveraging persistent server directories:

Layer 1 — Hardware:

Hardware Layer: AMD Radeon Pro W7900D (gfx1100) with 48 GB VRAM, running ROCm 7.2.1 on an Ubuntu container.

Layer 2 — Environment Management:

Shared persistent venv /persistent/amd_ai_env hosts the ROCm-exclusive PyTorch stack, shared by both ComfyUI and WebUI. Isolated venv /persistent/webui_env holds only WebUI lightweight dependencies, symlinking the shared torch stack to avoid duplicate installation. Persistent storage partition keeps models, scripts, and logs across container restarts.

Layer 3 — Application Services:

ComfyUI (port 8188): custom-node workflows, API batch generation, flexible model-chain debugging. A1111 WebUI (port 7860): visual txt2img interface with rapid parameter tuning.

Layer 4 — Remote Access:

SSH local port-forward tunnel maps server ports 7860/8188 to the local browser, enabling secure remote access without exposing services to the public internet.

Directory Layout:

```
/workspace/ComfyUI                      ComfyUI main program (port 8188)
/workspace/stable-diffusion-webui       A1111 WebUI main program (port 7860)
/persistent/amd_ai_env                  Shared ROCm torch env (torch, torchvision, torchaudio)
/persistent/webui_env                   WebUI venv (torch symlinked from amd_ai_env)
/persistent/comfyui/models              Model directory (checkpoints at top level)

ROCm-SDXL-Workbench/                    Organized deliverables
  scripts/    startup & deploy scripts
  patches/    sd_disable_initialization patch + AMD ROCm notes
  workflows/  SDXL txt2img ComfyUI workflow (API format)
  docs/       four Chinese deployment documents + English README
  demo/       generated images, terminal logs, hardware info, benchmark
  requirements_amd.txt   merged dependency list (203 pinned packages)
```

4. Model & Algorithm Introduction

Core Model:

Primary generation model: SDXL 1.0 Base by Stability AI (6.9 GB, 2515 tensors, sha256 31e35c80fc4829d14f90153f4c74cd59c90b779f6afe05a74cd6120b893f7e5b).

Secondary Model:

Animagine XL 4.0 (cagliostrolab) — 6.9 GB anime-focused model, auto-detected by ComfyUI via extra_model_paths.yaml.

Inference Components:

CLIP text encoder (open_clip 2.20.0 locked — 3.x breaks SDXL attention mask shape). SDXL dedicated VAE (embedded in CheckpointLoaderSimple output 2). Multi-step samplers: euler_ancestral / euler / dpmpp_2m, all ROCm-validated.

Proprietary Tooling:

Self-developed amd_vram_auto.py: auto-detects AMD VRAM capacity and recommends resolution and model-loading strategy to prevent OOM and inference stalls.

Exploratory:

Z-Image-Turbo lightweight generation model: workflow scaffolded and tested. VAE weight mismatch prevents full inference; integration is reserved as a future extension.

5. Adaptation Description for AMD Radeon GPU / ROCm

PyTorch底层适配:

Uninstalled CUDA PyTorch and deployed ROCm-exclusive PyTorch 2.9.1+rocm7.2.1 in a shared persistent venv, eliminating duplicate installations and ensuring both ComfyUI and WebUI use the identical ROCm build.

系统环境变量修复:

Startup scripts force-unset LD_LIBRARY_PATH before launching either service. This prevents torchvision._C.so from loading a stale system libtorch, which causes 'RuntimeError: operator torchvision::nms does not exist'. Exported HSA_OVERRIDE_GFX_VERSION=11.0.0 for gfx1100 architecture compatibility.

依赖版本锁定:

Pinned open-clip-torch==2.20.0 (3.x breaks SDXL attn_mask). Pinned k-diffusion to commit ab527a9 (master requires unavailable dctorch). Pinned transformers==4.40.2 with the sd_disable_initialization patch.

源码补丁 1 — transformers 离线 CLIP 加载:

Patch 1 (critical): modules/sd_disable_initialization.py — replaces the CLIP config string with an offline-constructed CLIPTextConfig object, preventing transformers from making a network request to huggingface.co/None (404) inside a background thread, which caused the WebUI to silently hang at 'Creating model from config'.

源码补丁 2 — VAE 预览模型离线化:

Patch 2: manually placed vaeapprox-sdxl.pt (sha256 6c8db511f910184436a613807ce06a729f4cf96bbf7977c7c6c40d5b3e4a8333) into models/VAE-approx/, bypassing the GitHub release download that is blocked in this environment.

AMD 显存调度优化:

Enabled async weight offloading (2 streams), pinned memory, and comfy-kitchen ROCm backends (hip, eager, cuda disabled). torch.backends.cudnn.enabled = False for AMD performance.

工具适配:

Reinstalled stable OpenCode AI terminal assistant for environment verification, batch command execution, and log parsing during deployment and troubleshooting.

6. Project Source Code

Complete source code is organized under ROCm-SDXL-Workbench/ in the submission repository.

The English README (ROCm-SDXL-Workbench/README.md) contains: environment configuration (shared amd_ai_env + isolated webui_env), startup commands for ComfyUI and WebUI, dependency list (203 pinned packages), known-error fixes (LD_LIBRARY_PATH, VAE invalid, transformers hang), SSH tunnel remote access guide, and benchmark results.

Key scripts: start_comfyui.sh, start_webui.sh, start_sshd.sh, full_amd_deploy.sh, restore_comfyui.sh, check_ssh_server_info.sh, sdxl_benchmark.py.

Patches: sd_disable_initialization.patch (git diff) + sd_disable_initialization.py (drop-in replacement) + detailed AMD ROCm compatibility notes.

Workflow: sdxl_base_1.0_workflow.json — a fully-tested ComfyUI API workflow for SDXL 1024×1024 txt2img (CheckpointLoaderSimple → CLIPTextEncode → KSampler → VAEDecode → SaveImage).

7. Benchmark & Demo Results

All benchmarks executed on AMD Radeon Pro W7900D (gfx1100, 48 GB VRAM), ROCm 7.2.1, PyTorch 2.9.1+rocm7.2.1, ComfyUI 0.29.0.

SDXL 1024×1024 Inference:

SDXL 1.0 Base, 1024×1024, 20 steps (euler_ancestral), cfg=7.0, measured via scripts/sdxl_benchmark.py against the live ComfyUI API.

- Run 1: 8.02 s | Peak VRAM: 31.20 GB
- Run 2: 8.02 s | Peak VRAM: 31.21 GB
- Run 3: 8.02 s | Peak VRAM: 31.21 GB
- Average: 8.02 s | Max peak VRAM: 31.21 GB | Idle (model loaded): ~21.98 GB

Hardware (rocm-smi):

Hardware identification via rocm-smi: GPU[0] = AMD Radeon Graphics (gfx1100), VRAM total 51522830336 B (~48 GB), temperature 23°C, power 14 W (idle).

Demo Images:

Generated demo images (1024×1024 SDXL):











Reproduction logs included in demo/logs/: error1_LD_LIBRARY_PATH_torchvision_nms.log (torch/torchvision ABI mismatch), error2_VAELoader_VAE_is_invalid.log (checkpoint without VAE weights), and fixed_sdxl_normal_generation.log (successful 8.02 s run).

8. Supplementary Materials

Complete submission package (sdxl_rocm_project.tar.gz, 310 MB) contains: source code (ComfyUI + WebUI, .git excluded), organized deliverables, demo assets, and logs. All private keys, passwords, and server-specific IPs have been redacted.

WebUI interface reference: the official A1111 WebUI screenshot is provided in demo/ as WebUI_interface_official.png.

Remote access is via SSH local port-forward tunnel (recommended) or Cloudflare Tunnel (for firewall-restricted environments). See README.md Section 7 for details.

Note: This project does not use any external API for inference. All model loading, sampling, and rendering happen locally on the AMD GPU. The environment is fully offline-capable once dependencies and models are in place.